Project Documentation

Implement Client Script and UI Policy (Incident)

Date	29 September 2026
Team ID	SWTID-2026-7872
Project Name	Implement Client Script and UI Policy (Incident)

1. Introduction

Project Title: Implement Client Script and UI Policy (Incident)

Team Members:

Name	Role
[Member 1]	Team Lead / Developer (UI Policies)
[Member 2]	Developer (Client Scripts)
[Member 3]	Developer (GlideAjax / Script Include)
[Member 4]	Tester / Documentation

2. Project Overview

Purpose: To improve the quality of incident data by validating input and controlling field behaviour directly on the ServiceNow Incident form. The goals are to reduce incomplete records, guide agents with clear messages and avoid rework.

Features:

- UI Policies that make fields mandatory, visible or read-only based on conditions (State, Priority, Contact type).
- Client Scripts: onLoad messages, onChange handling (Category/Subcategory, Priority, State, Caller), onSubmit validation and onCellEdit list validation.
- GlideAjax calls to a client-callable Script Include (VIP caller check, open incident count).
- Consistent field and error messages using the g_form API.
- Changes packaged in an Update Set for deployment.

3. Architecture

This project runs on the ServiceNow platform, so the layers of the template map to platform components instead of React, Node.js and MongoDB.

Frontend (client side): The Incident form in the browser. UI Policies and Client Scripts run here and use the g_form API to read and change fields, show messages and block submission.

Backend (server side): A client-callable Script Include (server-side JavaScript) answers GlideAjax requests. Optional Business Rules and Data Policies provide server-side enforcement.

Database: The Incident table (`incident`) in the ServiceNow database, accessed through GlideRecord. Key fields used: number, caller_id, short_description, description, category, subcategory, priority, state, assignment_group, close_code, close_notes, hold_reason.

4. Setup Instructions

Prerequisites:

- A ServiceNow Personal Developer Instance (PDI) from developer.servicenow.com.
- A user with the admin role on the instance.
- A modern web browser (e.g., Chrome) with developer tools.
- Basic knowledge of JavaScript.

Installation / configuration steps:

1. Sign up at developer.servicenow.com and request a PDI; log in as admin.
2. Create a Local Update Set (System Update Sets > Local Update Sets > New) and set it as current so all changes are captured.
3. Create UI Policies on the Incident table (System UI > UI Policies > New).
4. Create Client Scripts on the Incident table (System Definition > Client Scripts > New).
5. Create the Script Include with Client callable checked (System Definition > Script Includes > New).
6. Mark the Update Set complete and, if needed, export it to move the changes to another instance.

No environment variables are required. Instance URL and credentials are used only to log in and are never stored in scripts.

5. Folder Structure

ServiceNow stores configuration as records instead of files. The structure below shows the records created for this project.

Client (browser side):

```
Incident [incident]
|-- UI Policies (sys_ui_policy)
|     |-- Mandatory Short Description and Caller
|     |-- Resolved Requires Close Code and Notes
|     |-- Hide Resolution Fields Until Resolved
|     |-- Priority 1 Requires Assignment Group
|     `-- On Hold Requires Reason
`-- Client Scripts (sys_script_client)
      |-- onLoad  - Show form info message
      |-- onChange - Category (clear Subcategory)
      |-- onChange - Priority / State / Caller
      |-- onSubmit - Description validation
      `-- onCellEdit - State validation (list view)
```

Server (platform side):

```
Script Includes (sys_script_include)
`-- IncidentUtils (Client callable)
      |-- getCallerInfo()      -> VIP flag
      `-- getOpenIncidentCount()

Update Set: Incident Client Script and UI Policy
```

6. Running the Application

There is no local server to start. The configuration runs as soon as it is saved on the instance.

1. Log in to the instance and open Incident > Create New (the incident form).
2. Perform the test steps from the test cases (leave fields empty, change Category, set State to Resolved, and so on).
3. To test as another user, use Impersonate User from the user menu.
4. Use the browser console (F12) and the JavaScript Log / Script Debugger to troubleshoot client scripts.

7. API Documentation

The project exposes one client-callable Script Include method (used through GlideAjax) and relies on the platform Table API for the Incident table. Sample values below are illustrative.

7.1 GlideAjax: IncidentUtils.getCallerInfo

Item	Detail
Method	GlideAjax (client to server, asynchronous)
Script Include	IncidentUtils (Client callable)
Parameters	sysparm_name = getCallerInfo; sysparm_caller_id = sys_id of the caller

Item	Detail
Returns	JSON string with the VIP flag and open incident count

```
var ga = new GlideAjax('IncidentUtils');
ga.addParam('sysparm_name', 'getCallerInfo');
ga.addParam('sysparm_caller_id', g_form.getValue('caller_id'));
ga.getXMLAnswer(function(answer) {
    var info = JSON.parse(answer);
    if (info.vip) { g_form.addInfoMessage('VIP caller'); }
});

// Example response:
{ "vip": true, "openIncidents": 3 }
```

7.2 Table API (Incident)

Method	Endpoint	Parameters / Body	Example Response
GET	/api/now/table/incident	sysparm_limit, sysparm_query, sysparm_fields	200 OK: {"result": [{"number": "INC0010001", ...}]}
POST	/api/now/table/incident	JSON body: caller_id, short_description, description, category	201 Created: {"result": {"number": "INC0010002", "sys_id": "..."}}
PATCH	/api/now/table/incident/{sys_id}	JSON body with fields to update	200 OK: {"result": {...updated record...}}

Note: UI Policies and Client Scripts run only in the browser form. Records created through the Table API bypass them, so equivalent Business Rules or Data Policies are recommended for server-side enforcement.

8. Authentication

- Users log in to the instance with local credentials or SSO (SAML/OAuth) configured by the platform.
- After login the browser keeps a session cookie; client-side calls such as GlideAjax carry the session and a CSRF token (g_ck) automatically.
- Authorization uses roles and Access Control Lists (ACLs) on tables and fields, so scripts only act on what the logged-in user may access.
- The Script Include is marked client callable and should check roles where needed (for example gs.hasRole).
- REST/Table API calls use Basic authentication or OAuth 2.0 tokens with an integration user.

9. User Interface

Incident form with mandatory fields highlighted (UI Policy)

[Insert screenshot]

Resolution fields shown only when State is Resolved

[Insert screenshot]

Client script messages (onChange / onSubmit)

[Insert screenshot]

10. Testing

Strategy: functional testing of each UI Policy and Client Script, performance testing of form load and GlideAjax response, and user acceptance testing (UAT) by service desk agents.

- **Functional (FT-01 to FT-04):** mandatory fields, conditional mandatory fields, onChange behaviour, onSubmit validation.
- **Performance (PT-01 to PT-03):** form load under 3 seconds, GlideAjax response, concurrent form loads.
- **UAT (TC-001 to TC-008):** positive and negative scenarios with bug tracking and sign-off.
- **Tools:** browser developer tools, Script Debugger, Impersonate User; the Automated Test Framework (ATF) can be added for regression tests.

11. Screenshots or Demo

Screenshot: UI Policy configuration record

[Insert screenshot]

Screenshot: Client Script configuration record

[Insert screenshot]

Demo link: [add demo video link]

12. Known Issues

Defects found during testing are recorded in the Bug Tracking table of the UAT document. Known limitations of the approach:

- Client-side rules do not apply to records created by import sets, integrations or the Table API.
- Many scripts on one form can increase load time.
- UI Policies and Client Scripts that act on the same field depend on execution order and need careful testing.

13. Future Enhancements

- Add Business Rules and Data Policies for server-side enforcement.
- Extend the validations to Problem, Change and Request forms and to Catalog Client Scripts.
- Add automated regression tests with the Automated Test Framework.
- Create dashboards to track data-quality improvement.